Module 3: Face Recognition

Face Detection vs Face Recognition

Face Detection:

- Finding faces in an image
- Output: Bounding boxes (x, y, width, height)
- Libraries: MediaPipe, OpenCV, dlib

Face Recognition:

- Identifying WHO the face belongs to
- Requires prior enrollment (known faces)
- Two types:
  - 1:1 Verification: 'Is this person X?'
  - 1:N Identification: 'Who is this person?'
- For attendance: We use 1:1 Verification

Feature Extraction & Embeddings

Embedding: A numerical representation of a face

- Finding faces in an image
- Typically 128 or 512-dimensional vector
- Similar faces have similar embeddings

How it works:

- Neural network trained on millions of faces
- Maps face image to fixed-size vector
- Captures identity, ignores lighting/angle/expression

Comparison:

- Euclidean distance or Cosine similarity
- Small distance = same person
- Threshold determines match/no-match

Matching Thresholds & Error Rates

Two types of errors:

- False Accept (FA): Wrong person accepted
- False Reject (FR): Right person rejected

Error rates:

- FAR (False Accept Rate): Security risk
- FRR (False Reject Rate): User inconvenience

Trade-off:

- Lower threshold = more false accepts
- Higher threshold = more false rejects

For attendance:

- Threshold ~0.70 balances security & usability
- Flag uncertain matches for manual review

Liveness Detection Principles

Problem: Photos and videos can fool basic recognition

Liveness = Proving a real person is present

Techniques:

- Passive: 3D depth, texture analysis, reflection
- Active: Challenges (blink, smile, turn head)

MediaPipe Face Mesh:

- Detects 468 3D facial landmarks
- Real faces have depth variation
- Printed photos are flat

Challenge-response:

- Random action required
- Prevents replay attacks

Privacy-Preserving Biometrics

NEVER store raw face images

- Images can be misused (deepfakes, tracking)
- Data breach = permanent identity compromise

What can we do instead?

Approach 1: Storing Embeddings Directly

Produce a face embedding with a pre-trained neural network

- Such as FaceNet or ArcFace
- Captures identity-relevant features of a face without storing the visual image
- Compare the similarity of embeddings for identify verification.

A big improvement over storing raw images, but face embeddings can be reversed.

- Tools like Arc2Face and IdDecoder can reconstruct recognizable face images from embeddings alone
- So embeddings are not fully safe.

A simple example using the face_recognition library

```
import face_recognition

image = face_recognition.load_image_file("student.jpg")
embedding = face_recognition.face_encodings(image)[0]

# Store 'embedding' in your database (a 128-d NumPy array)
# Then delete the original image immediately
```

Approach 2: Hashing Embeddings with SHA-256

Hash each embedding with SHA-256, similar as hashing a password

- SHA-256 is completely non-reversible: attackers cannot recover the original input from the hash output – perfect for privacy.
- Problem: SHA-256 has the avalanche effect – a tiny change in input produces a completely different output.
- Two photos of the same person taken at different moments produce slightly different embeddings due to lighting, angle and expression, but their hash values are totally different, making it infeasible to tell they are the same person.

```
import hashlib, numpy as np

embedding_1 = np.array([0.12, -0.45, 0.78, ...])  # photo A
embedding_2 = embedding_1 + 0.001  # photo B (tiny difference)

hash_1 = hashlib.sha256(embedding_1.tobytes()).hexdigest()
hash_2 = hashlib.sha256(embedding_2.tobytes()).hexdigest()

print(hash_1 == hash_2)  # False! Completely different hashes
# No way to measure how 'close' the two hashes are
```

Approach 3: SimHash (Locality-Sensitive Hashing)

Locality-Sensitive Hashing (LSH)

- Similar inputs produce similar outputs.
- The most common variant for numerical vectors is called SimHash

Idea

- Generate a set of random vectors (called hyperplanes).
- For each one, compute the dot product with the face embedding and record only the sign: positive becomes 1, negative becomes 0.
- String all the sign bits together into a binary code.
- Two embeddings from same person will land on the same side of most hyperplanes, so their codes will differ in only a few bits. Different people will differ in many bits.
- To compare two SimHash codes, count the number of differing bits (the Hamming distance). A small Hamming distance means the original embeddings were similar—likely the same person. A large Hamming distance means different people.

Approach 3: SimHash (Locality-Sensitive Hashing)

A minimal implementation

```
import numpy as np

# Generate random hyperplanes (do this once, keep them fixed)
np.random.seed(42)
planes = np.random.randn(64, 128)  # 64 bits, 128-d embeddings

def simhash(embedding):
    projections = planes @ embedding
    bits = "".join("1" if p > 0 else "0" for p in projections)
    return bits

def hamming_distance(h1, h2):
    return sum(a != b for a, b in zip(h1, h2))

# Enrollment: hash and store
stored_hash = simhash(enrolled_embedding)

# Verification: hash new photo and compare
new_hash = simhash(new_photo_embedding)
dist = hamming_distance(stored_hash, new_hash)
is_match = dist < 12  # tune this threshold for your system
```

Approach 3: SimHash (Locality-Sensitive Hashing)

Why is this private?

- Each bit only records which side of a random hyperplane the embedding falls on.
- With 64 bits from a 128-dimensional vector, the system is heavily underdetermined—infinitely many different embeddings produce the same hash. An attacker cannot reconstruct the original embedding from the hash.

Why does matching still work?

- There is a well-defined mathematical relationship between Hamming distance and cosine similarity. You can estimate how similar the original embeddings were just by comparing the hashes, without ever seeing the raw vectors.
- You can tune two parameters: the number of hash bits (more bits means finer discrimination but more risk of false rejects) and the Hamming distance threshold (lower means stricter matching). For 128-dimensional embeddings, 64 bits with a threshold around 10–15 is a reasonable starting point.

Beyond SimHash

- Techniques like BioHashing, fuzzy commitment schemes, and cancelable biometrics provide stronger guarantees around revocability and cryptographic security.
- These are great directions for extending your capstone work.

Risk Scoring Integration

Combine multiple signals into a risk score:

- Face match score (0-1)
- Liveness score (0-1)
- Location accuracy
- Device trust level
- Time of check-in vs. session time

Risk levels:

- LOW (< 0.3): Auto-approve
- MEDIUM (0.3-0.6): Approve with flag
- HIGH (0.6-0.8): Requires review
- CRITICAL (> 0.8): Auto-reject

Instructor reviews flagged check-ins